ThreatChain — Decentralized Threat Intelligence Sharing on Blockchain

Graduation Project Proposal

Student: [Student Name] | **Supervisor:** [Supervisor] | **Date:** May 2026

1. Problem Statement

Cybersecurity organizations — banks, national CERTs, ISPs, hospitals — operate in silos. When one organization discovers a new threat (a malware family, a phishing campaign, or a previously unknown vulnerability) the information typically remains inside that organization. Other organizations face the same threat days or weeks later, blind. Existing threat-intelligence platforms, whether commercial feeds or central CERT databases, introduce a trust problem and a single point of failure: whoever owns the central server can withhold, alter, or lose alerts, and the central server itself becomes a high-value target. There is no widely deployed mechanism that lets multiple organizations share threat intelligence with cryptographic proof of origin, immutability, and no central owner.

2. Proposed Solution — ThreatChain

ThreatChain is a peer-to-peer, blockchain-based platform on which verified organizations submit cryptographically signed threat intelligence: indicators of compromise such as malicious IP addresses, domains, file hashes, and phishing URLs. Every node holds a full copy of the chain. Every submission is signed with the submitter's private key, sealed into a block via consensus, and gossiped to all peers. Tampering with any block invalidates every block after it, and every peer rejects an invalid chain. The result is a shared, tamper-evident, multi-organization threat-intelligence ledger with no single owner and no single point of failure.

3. Why Blockchain

- **No central owner** — eliminates the single point of failure and the trust problem inherent in centralized feeds.
- **Cryptographic signatures** — every report is provably attributed to its originating organization and cannot be forged.
- **Hash-chained blocks** — tampering with any record is immediately visible to every node, because the chain of hashes breaks.
- **Chronological proof** — the ledger establishes when a threat was first reported, supporting accountability and SLA enforcement between participating organizations.

4. System Architecture

- **Blockchain Core** — custom Python implementation with SHA-256 hash chaining and Proof-of-Work consensus.
- **Cryptographic Identity** — each organization owns a 2048-bit RSA keypair. The public key is the organization's on-chain identity; the private key signs every submission via RSA-PSS-SHA256.
- **P2P Network** — nodes discover each other via peer registration, exchange chains over HTTP, and adopt the longest valid chain.
- **Persistence** — JSON-based per-node storage of the chain, the organization registry, and the peer list.
- **Web Interface** — Flask + HTML/CSS/JavaScript dashboard providing registration, threat submission, a live feed, a blockchain explorer, and network/peer status.

5. Implementation Status

A working prototype is complete. Three independent nodes can run on the same machine or across a LAN, each with its own data folder. The end-to-end flow has been verified: organization registration generates the keypair; threat submission signs the transaction and mines a new block (typical Proof-of-Work time under one second at difficulty 4); the transaction is broadcast to peers; signatures are independently verified on each peer; and the longest-valid-chain rule resolves divergence between nodes. A tamper test has also been demonstrated: corrupting a local *chain.json* file marks the chain invalid, and triggering consensus synchronization from peers automatically restores the valid copy.

6. Scope and Deliverables

- Working multi-node prototype (Python, Flask, HTML/CSS/JavaScript).
- Academic report covering motivation, related work, design, implementation, and evaluation.
- Live demonstration including the tamper-and-recovery test.
- Written discussion of limitations and future work.

7. Limitations and Future Work

The current prototype uses Proof-of-Work for educational clarity. A production deployment would adopt Proof-of-Authority with permissioned validators (for example, Jordan CERT alongside sector regulators). Privacy of sensitive intelligence is addressed by introducing Traffic Light Protocol (TLP) tiers, with amber and red intelligence stored as off-chain encrypted payloads referenced by on-chain metadata. Sybil resistance is provided by an out-of-band organization-verification gateway that issues signing certificates only after the requesting organization is validated. These are documented design extensions rather than unresolved problems.

8. Discussion Points for Supervisor

- **Scope confirmation** — is the multi-node software prototype plus academic report sufficient for the graduation requirement, or should a hardware/multi-machine deployment be attempted within the project timeframe?
- **Related work** — the student is aware of the existing literature on blockchain-based threat-intelligence sharing and welcomes specific recommendations on which papers to cite as primary references.
- **Optional extensions** — whether to implement Proof-of-Authority, TLP tiers, or the organization-verification gateway within the project timeframe, or to document them in the Future Work section only.